AUTHOR QUERIES

AUTHOR PLEASE ANSWER ALL QUERIES

PLEASE NOTE: We cannot accept new source files as corrections for your article. If possible, please annotate the PDF proof we have sent you with your corrections and upload it via the Author Gateway. Alternatively, you may send us your corrections in list format. You may also upload revised graphics via the Author Gateway.

Carefully check the page proofs (and coordinate with all authors); additional changes or updates WILL NOT be accepted after the article is published online/print in its final form. Please check author names and affiliations, funding, as well as the overall article for any errors prior to sending in your author proof corrections. Your article has been peer reviewed, accepted as final, and sent in to IEEE. No text changes have been made to the main part of the article as dictated by the editorial level of service for your publication.

Authors will be charged an overlength fee for articles in excess of 8 pages.

AQ:1 = Please confirm or add details for any funding or financial support for the research of this article.

AQ:2 = Please provide the expansions of the acronyms OPEVA, JU, BPI, and AI for your funding agency. Providing the correct acknowledgment will ensure proper credit to the funder.

AQ:3 = Please confirm whether the corresponding author has been identified correctly.

AQ:4 = Please confirm whether the edits made in the current affiliation of all the authors are correct.

AQ:5 = Please provide the postal code for German University of Technology in Oman (GUtech), Télécom SudParis, and La Rochelle University.

Split Multi-Task Federated Learning for Battery Health and Capacity Estimation in Electric Vehicles

Abdelaziz Amara Korba, *Member, IEEE*, Mouhamed Amine Bouchiha, *Member, IEEE*, and Yacine Ghamri-Doudane, *Senior Member, IEEE*

***Abstract*— The rapid adoption of electric vehicles (EVs) has heightened the need for advanced diagnostic frameworks to ensure the safety, reliability, and longevity of battery systems. Batteries, as the cornerstone of EV performance, are susceptible to capacity degradation, thermal instability, and anomalies that compromise efficiency and safety. Traditional diagnostic approaches, relying on physics-based models and heuristic thresholds, often fail to address the complexity and variability of real-world EV operations, necessitating scalable, efficient, and privacy-preserving solutions. This paper presents a Split Multi-Task Federated Learning (SMTFL) framework that leverages a Transformer-based architecture to address the dual challenges of battery health anomaly detection and capacity estimation. By combining the efficiency of split learning with the collaborative benefits of federated learning, the proposed system enables EVs to train and share partial model updates while retaining sensitive data locally. Experimental results on a recent large-scale EV dataset demonstrate the superior performance of the SMTFL framework, achieving an accuracy of 98.9% for battery health anomaly detection and a mean absolute error (MAE) of 0.32 Ah for capacity estimation, corresponding to a relative error of approximately 1.80%, significantly outperforming state-of-the-art approaches. By training a single model for both tasks and performing computations during charging sessions, the framework minimizes energy consumption and computational overhead, ensuring practicality for large-scale deployment in real-world EV systems.**

***Index Terms*— Electric vehicle, battery state of health, state of charge (SOC), battery management system (BMS), split federated learning, transformer.**

I. Introduction

THE widespread adoption of electric vehicles (EVs) has brought new challenges in ensuring the safety, reliability, and performance of their battery systems. As the central energy source, batteries directly impact key aspects such as vehicle range, lifespan, and overall efficiency. However, they remain highly susceptible to issues such as capacity degradation, thermal instability, and lithium plating, which can compromise both safety and performance. These challenges highlight the urgent need for advanced diagnostic frameworks capable of accurately monitoring battery health and estimating remaining capacity under diverse and dynamic real-world conditions.

In this work, we distinguish between battery health anomaly detection and capacity estimation, two related yet distinct diagnostic tasks. Following established battery diagnostics conventions [1], State of Health (SoH) refers to the overall degradation of the battery (often expressed as a percentage of initial capacity), whereas capacity estimation quantifies the absolute remaining charge in ampere-hours (Ah). Accordingly, we frame health monitoring as a binary anomaly detection task (normal vs. anomalous conditions such as lithium plating or thermal runaway), and capacity estimation as a regression task.

The complexity of battery diagnostics stems from the highly variable operating environments of EVs [2], which involve fluctuating charging and discharging cycles, environmental conditions, and aging effects. Traditional methods, such as physics-based models or predefined threshold-based diagnostics, struggle to adapt to such real-world complexities, often leading to suboptimal performance. Moreover, the integration of EVs into connected vehicular networks introduces additional constraints, including the need for scalable solutions that can handle large data volumes while preserving data privacy.

Machine learning has emerged as a powerful tool for battery diagnostics, leveraging multivariate time-series data to identify anomalies and predict capacity trends [2], [3], [4]. However, deploying these models in distributed electric vehicle (EV) fleets presents several critical challenges. EVs, with their limited processing power and memory, cannot feasibly train or host complex machine learning models locally. Furthermore, transferring raw battery data to centralized servers raises significant privacy concerns, particularly within large-scale vehicular networks. This necessitates solutions that safeguard sensitive information while enabling effective model training. The variability in battery usage patterns and operational conditions across EVs further complicates the creation of generalized models, as the heterogeneous data demands robust approaches to maintain predictive accuracy. Additionally, the exponential growth in the number of EVs calls for scalable frameworks capable of efficiently handling model updates and

Received 29 April 2025; revised 26 December 2025 and 12 January 2026; accepted 23 January 2026. This work was supported in part by the OPEVA Project funded by Chips Joint Undertaking (Chips JU) from European Union's Horizon Europe Programme and the National Authorities (France, Czechia, Italy, Portugal, Türkiye, and Switzerland) under Grant 101097267; and in part by the Project funded by BPI France through France 2030 Program on "Embedded AI," France. *(Corresponding author: Abdelaziz Amara Korba.)*

Abdelaziz Amara Korba is with the Department of Computer Science, German University of Technology in Oman (GUtech), Muscat, Oman (e-mail: abdelaziz.amarakorba@gutech.edu.om).

Mouhamed Amine Bouchiha is with SAMOVAR, Télécom SudParis, Paris, France (e-mail: mbouchiha@telecom-sudparis.eu).

Yacine Ghamri-Doudane is with L3i, La Rochelle University, La Rochelle, France (e-mail: yacine.ghamri@univ-lr.fr).

D

1558-4127 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

managing communication overhead between EV clients and parameter servers. To be practical, such solutions must also align with existing EV infrastructure, exploiting idle periods, such as charging sessions, for computational tasks without compromising the energy and operational requirements of the EVs.

To address these challenges, this paper introduces a Split Federated Learning framework combined with a multi-task transformer-based model designed to address two critical tasks: battery health anomaly detection and capacity estimation. The framework uses a unified approach, leveraging a single model for both tasks, which simplifies deployment and reduces operational complexity. This approach eliminates the need for training and maintaining separate models for each task, resulting in significant practical and computational benefits.

The proposed framework employs a hierarchical federated learning architecture that integrates EVs, edge servers at charging stations, and a global server in the cloud. By using federated learning principles, sensitive data remains localized at the EV level, safeguarding privacy while enabling collaborative training across distributed nodes. Furthermore, the hierarchical design coordinates learning across multiple levels, ensuring model consistency and scalability. The use of split learning further enhances the system's efficiency by dividing the model into segments, with only the initial layers being trained locally on the EV, while the rest are processed on the edge server. This design minimizes computational demands on EVs, making the framework suitable for resource-constrained devices. Another critical advantage of this approach is the optimization of training sessions during the EV's charging periods. By performing training while the EV is connected to the charging station, the framework avoids depleting the battery for computational tasks, preserving energy for essential functionalities. These design considerations make the proposed solution practical and efficient, aligning with the energy and privacy requirements of modern EV systems.

To summarize, the main contributions of this paper are:

- Developing a single model capable of performing both battery health and capacity estimation. This unified approach, facilitated by split learning, simplifies deployment, reduces complexity, and enhances energy efficiency by allowing EVs to train only a portion of the model locally while offloading the remaining computations to the edge server. This minimizes computational demands on EVs and ensures scalability for real-world applications.
- Introducing a privacy-preserving and scalable federated learning architecture that integrates differential privacy mechanisms to protect sensitive data. The framework facilitates seamless collaboration among EVs, edge servers at charging stations, and a global parameter server, ensuring efficient training across EVs while maintaining model consistency and safeguarding sensitive data within the EVs.
- Demonstrating the practicality and effectiveness of the proposed solution through rigorous validation on a recent and realistic, large-scale EV dataset, achieving superior performance over state-of-the-art methods for both battery health anomaly detection and capacity estimation tasks.

The remainder of this paper is organized as follows: Section II reviews the related work. Section III details the development and operation of the proposed framework. Section IV presents the performance evaluation results along with a comparative analysis against state-of-the-art (SOTA) approaches. Finally, Section V concludes the paper.

II. Related Work

Battery diagnostics and capacity estimation are critical research areas for ensuring the safety, reliability, and operational efficiency of electric vehicles (EVs). Early approaches often relied on statistical models and traditional machine learning techniques, such as statistical distribution analysis and canonical algorithms, to diagnose faults and detect anomalies [5], [6]. More recently, Li et al. [7] review artificial neural network (ANN) approaches for state-of-health (SoH) estimation and remaining useful life (RUL) prediction, discussing battery aging mechanisms, available datasets, and ANN architectures, and outlining key challenges related to deployment feasibility and model interpretability.

Sultan et al. [8] propose an active cell balancing system to address imbalanced state of charge (SoC) in EV lithium-ion battery packs caused by cell inconsistencies. The study evaluates seven machine learning models for predicting battery lifespan and remaining useful life (RUL). Das et al. [9] estimate the state of health (SoH) of lithium-ion batteries in electric vehicles using machine learning algorithms like KNN, SVR, DT, and RF, based on direct battery measurements (voltage, current, temperature). The study proposes collaboration with hardware manufacturers for better battery health monitoring systems.

The advent of deep learning introduced advanced models [10], such as long short-term memory (LSTM) networks, capable of capturing temporal dependencies and improving anomaly detection in battery systems [6], [11]. However, the development of robust deep learning methods hinges on the availability of large-scale, publicly accessible datasets. In response, recent advancements in EV battery research have produced datasets tailored to various tasks, including anomaly detection, capacity estimation, and remaining useful life (RUL) prediction. For instance, datasets such as Sultan et al. [8] and HUST [12] focus on controlled experimental conditions, while large-scale datasets like EVBattery [1] provide diverse real-world charging data. Additionally, specialized datasets address manufacturing defects through CT scans [13] or explore capacity estimation using voltage relaxation profiles [14].

Xiong et al. [15] propose a lithium-ion battery health prognosis method using real-world BMS in electric vehicles. The approach combines partial charge voltage curves as health indicators with a moving-window-based RUL prediction model. While the method is well-suited for many applications, its reliance on linear degradation models may present challenges when applied to batteries exhibiting complex degradation patterns.

Deng et al. [16] propose a rapid health estimation framework for in-service EV battery packs that combines a digital-twin model based on an equivalent-circuit representation, feature extraction from short charging segments, and deep convolutional neural networks with transfer learning and domain adaptation to cope with limited labeled field data. Han et al. [17] emphasize the significance of dynamically reconfigurable battery management systems (RBSs) for enhancing fault tolerance, improving charge and temperature balancing, and achieving greater energy efficiency and operational flexibility. While the proposed approach offers valuable advancements, the implementation of RBSs may encounter challenges such as the computational demands of optimization algorithms, increased hardware design requirements, and practical considerations like managing transient behaviors during switch operations.

Liu et al. [18] tackle the challenge of battery state of health (SoH) estimation under limited sample conditions by introducing a refined composite multiscale discrete sine entropy (RCMDSE) algorithm. Similarly, Kumari et al. [19] propose a hybrid approach for estimating state of charge, health, and temperature (SoC, SoH, SoT). Their method combines an emperor penguin-based residual network-110 (R110) with bidirectional long short-term memory (BLSTM) networks. This approach demonstrates significant promise, achieving high accuracy on EV drive cycles simulated in MATLAB. However, the reliance on advanced optimization techniques, such as simulated annealing, may limit its real-time applicability in resource-constrained environments. Kant et al., [20] proposed a method that integrates a bidirectional recurrent neural network and long-short-term memory (biRNN-LSTM) to improve the prediction of the state of health (SoH) by identifying complex patterns and improving accuracy.

Despite notable advancements in battery diagnostics and capacity estimation, several limitations persist within the state of the art. Approaches targeting complex degradation behaviors often fail to capture nonlinear dynamics [8], while recent innovations [17], [19] face challenges related to computational overhead, limiting real-time applicability in resource-constrained EV. Many methods rely on datasets generated under controlled laboratory conditions [6], [11], restricting their generalizability to diverse real-world scenarios. Moreover, existing techniques often assume that EV data can be centrally collected, overlooking critical privacy concerns.

Federated learning (FL) has recently gained traction as a distributed training paradigm in a variety of application domains. For instance, Wu et al. [21] employ federated deep reinforcement learning to perform multi-objective optimization in STAR-RIS-aided satellite–aerial–ground integrated networks (SAGINs). In the context of battery health monitoring and state-of-health (SoH) estimation, several studies [22], [23], [24] have leveraged FL-based designs. Abbaraju and Kundu [22] propose a federated ensemble learning (FEL) approach that combines federated learning and ensemble learning to enhance SoH estimation. Their solution relies on an LSTM model that is updated after each trip, leveraging heterogeneous data sources such as traffic conditions and weather. Wang et al. [23] introduce an adaptive multi-personalized federated learning (AMFL) algorithm that refines SoH estimation by dynamically adjusting local models through personalized layers and client clustering while incorporating a differential privacy mechanism. Similarly, Wong et al. [24] develop a decentralized federated learning approach for Li-ion battery SoH forecasting, featuring an improved client selection strategy and real-world validation experiments.

Despite notable progress, existing battery diagnostic methods exhibit several limitations. Model-based approaches often rely on simplified linear degradation assumptions [15], failing to capture complex real-world aging dynamics. Federated learning (FL) methods for SoH estimation [22], [23], [24] reduce data centralization but impose significant computational burdens on resource-constrained EVs and lack energy-efficient training schedules, which could impact battery autonomy. Moreover, they typically focus on a single task (e.g., SoH estimation), requiring separate models for different diagnostics and increasing system complexity. Many methods are also evaluated on controlled laboratory datasets [6], [11], limiting their applicability to diverse real-world conditions.

To address these gaps, we propose a Split Multi-Task Federated Learning (SMTFL) framework that: (1) employs a Transformer-based encoder to model nonlinear temporal dependencies; (2) reduces on-device computation by offloading heavy layers to edge infrastructure via split learning, where only early layers are trained locally; (3) incorporates differential privacy mechanisms to protect sensitive battery data during model updates; (4) unifies battery health anomaly detection and capacity estimation into a single model, eliminating the need for separate, task-specific models and optimizing parameter efficiency; (5) schedules training exclusively during charging sessions to preserve EV battery life; and (6) is validated on the large-scale real-world EVBattery dataset [1] to ensure practical relevance. This innovative framework effectively balances scalability, privacy preservation, and computational efficiency, addressing the key challenges identified in the state of the art.

III. SMTFL Framework

Our framework integrates split learning and federated architectures for battery health monitoring, balancing local computation with global performance while incorporating differential privacy. The following sections detail the architectural components, including the split Transformer model, privacy-preserving training protocol, and federated coordination strategy.

A. System Model

The proposed system employs a hierarchical federated learning architecture that integrates three primary components: electric vehicles (EVs) as clients, edge servers deployed at charging stations, and a global parameter server in the cloud. The workflow is designed to ensure synchronization, prevent version conflicts, and facilitate efficient and scalable model training. Figure 1 illustrates the interactions and hierarchical structure among these components. At the start of each global

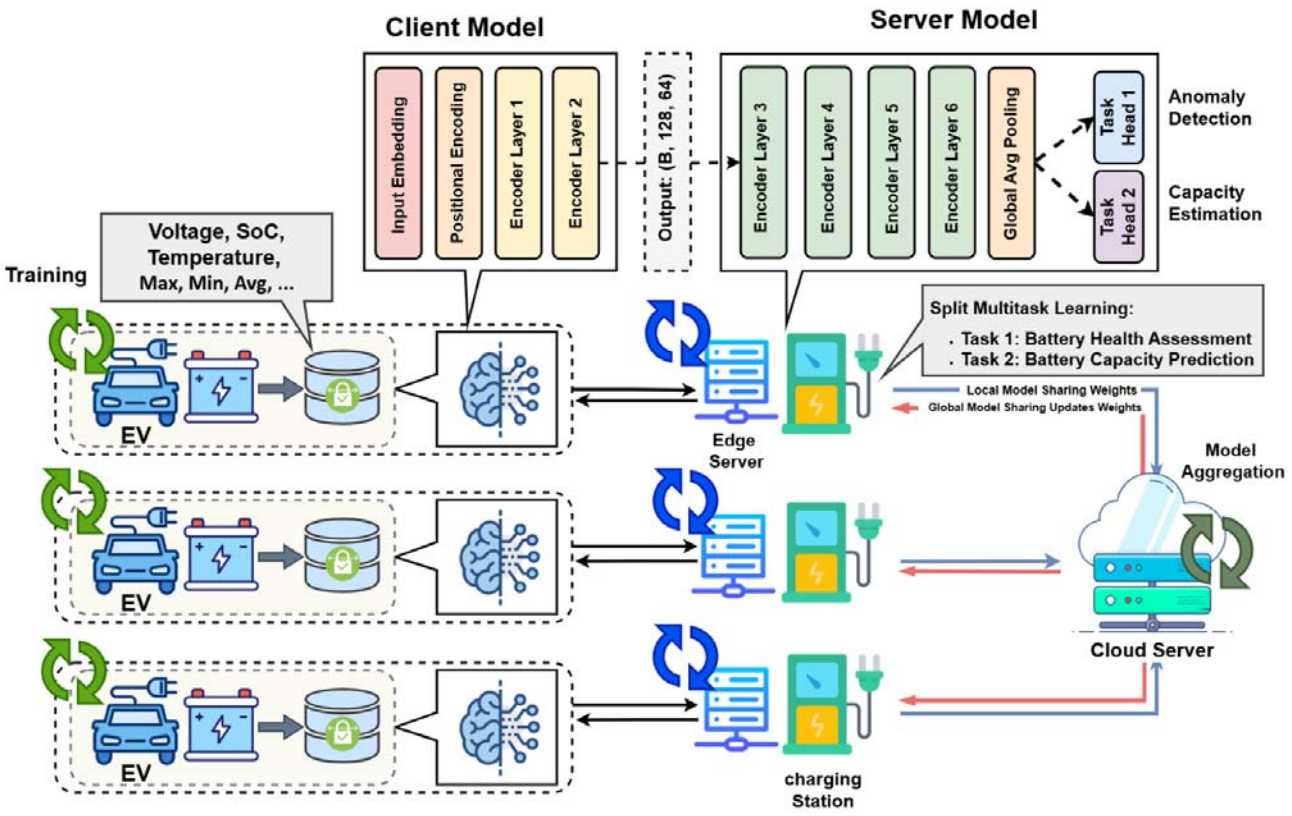


Fig. 1. System design. Top: the split Transformer architecture with lightweight local layers on EVs and heavier layers on edge servers. Bottom: hierarchical aggregation where edge servers aggregate local updates from multiple EVs, and a cloud server aggregates edge updates for global model refinement.

round, the cloud server distributes the global model to edge servers located within charging stations, which then propagate it to connected EVs. Each edge server collects model updates from EVs during a station-level Aggregation Time Window (ATW), performs local aggregation, and forwards the result to the cloud server. The cloud operates within a longer Synchronization Window (SW) to gather updates from all edge servers, execute global aggregation, and redistribute the refined model.

The SMTFL framework leverages Power Line Communication (PLC) over the charging cable, standardized in ISO 15118 [25], to transmit model updates and intermediate representations (smashed data) between EVs and edge servers. This approach eliminates separate communication hardware, reduces system complexity, and enhances security by utilizing the naturally isolated channel of the charging cable. The physical connection serves a dual purpose: power transfer and data exchange. Communication is secured with mutual Transport Layer Security (TLS) 1.3 [26], ensuring confidentiality, integrity, and authenticity. Charging stations communicate with the cloud server via standard Internet connections (e.g., fiber, 4G/5G), enabling reliable transmission of aggregated updates and global model distribution.

1) EV Participation in FL Rounds: To accurately estimate the number of EVs connected to a charging station during a given Aggregation Time Window (ATW), we propose a mathematical model incorporating key variables: the arrival rate of EVs (λ_{EV}), the average session duration ($T_{session}$), and the station's capacity (N_{max}). The goal is to determine the number of EVs utilizing the station within the ATW while accounting for constraints such as the limited number of charging points and varying levels of station utilization.

The arrival of EVs at a charging station is modeled as a Poisson process with average arrival rate λ_{EV} (EVs per unit time). During an ATW of duration T_{ATW}, the total expected arrivals is $N_{arrivals} = \lambda_{EV} \cdot T_{ATW}$. The actual number of EVs simultaneously connected during the ATW is influenced by $T_{session}$, N_{max}, and an occupation factor ρ (fraction of charging points typically in use):

$$N_{connected} = \min\left(\rho \cdot \frac{T_{session}}{T_{ATW}} \cdot N_{arrivals}, N_{max}\right).$$

When the station is underutilized, $N_{connected}$ is limited by arrival rate and session duration; when saturated, it is capped by N_{max}. The total number of EVs utilizing the station during the ATW, denoted N_{EV}, combines these effects:

$$N_{EV} = \min\left(\lambda_{EV} \cdot T_{ATW}, N_{max} \cdot \frac{T_{ATW}}{T_{session}}\right).$$

This formulation ensures the total is bounded by both arrival rate and station capacity for concurrent charging sessions.

In practice, EVs may not complete local training or transmit updates simultaneously within an ATW due to varying computational loads or intermittent connectivity. The edge server aggregates all updates received by the window's deadline; EVs that miss the cutoff participate in the subsequent ATW.

The subsequent SW at the cloud level provides additional buffering, allowing edge servers to forward aggregated models without requiring strict synchronization. Moreover, if an EV disconnects prematurely, it can resume training in a later session by downloading the latest global model. This design ensures robust operation under realistic, asynchronous update patterns and connectivity variations, making the system practical for real-world deployment.

2) EV-Level Operations: When an EV connects to a charging station, it downloads the client-side portion (first two Transformer layers) of the global model. Using local battery data, the EV performs forward and backward passes in coordination with the edge server to update its local layers. Throughout this process, differential privacy mechanisms are applied to both activations and weight updates. After completing local training, the EV transmits its updated and privacy-preserving client-side weights to the edge server for aggregation.

3) Edge-Level Operations: Each charging station is equipped with an edge server that coordinates local training and performs intermediate aggregation. Operating within a fixed Aggregation Time Window (ATW), the edge server collects and aggregates client-side weight updates from connected EVs. If no EVs connect during an ATW, aggregation is skipped to conserve resources. The edge server also trains the server-side layers using smashed data received from EVs. Once the ATW ends, the edge server transmits the locally updated model (aggregated client-side weights and updated server-side layers) to the global server during the Synchronization Window (SW).

4) Cloud-Level Operations: At the cloud level, the server acts as a global parameter server, operating within a longer SW to ensure all edge servers have sufficient time to transmit their aggregated models. The cloud server collects and integrates model updates from all active edge servers, performs global aggregation, and then redistributes the updated global model back to the charging stations. Stations subsequently provide the updated model to EVs upon their next connections, closing the federated learning loop.

B. Prediction Model Architecture

The prediction model employs a Transformer encoder to process multivariate time-series data, performing two critical tasks in parallel: battery health anomaly detection (binary classification) and capacity estimation (regression). The Transformer encoder is selected for its superior ability to capture long-range temporal dependencies, which are essential for accurate battery diagnostics. The model comprises six encoder layers, each containing a multi-head self-attention mechanism and a feed-forward network. In our split learning configuration, the architecture is divided at a precise boundary: the first two encoder layers form the client-side (EV) component, while the remaining four layers, along with global pooling and task-specific heads, constitute the server-side (edge) component. This split follows established privacy–complexity trade-offs in split learning [27]: early layers extract low-level features with minimal privacy risk, while deeper, computationally intensive layers are offloaded to the edge server. During operation, the EV processes raw data through the first two layers, producing intermediate activations (smashed data) of dimension `[batch_size, sequence_length=128, d_model=64]`, which are transmitted to the edge server.

We now detail the model components: input representation, Transformer encoder, and task-specific heads.

1) Input Representation: The input sequence $x \in \mathbb{R}^{T\times d}$, where T is the sequence length and d is the number of features, is projected into a higher-dimensional latent space using a trainable embedding layer:

$$x' = W_{\text{embed}} \cdot x + b_{\text{embed}}, \tag{1}$$

where $W_{\text{embed}} \in \mathbb{R}^{d\times d_{\text{model}}}$ and $b_{\text{embed}} \in \mathbb{R}^{d_{\text{model}}}$.

To encode temporal dependencies, a positional encoding is added:

$$\begin{aligned} \text{PE}(t, 2i) &= \sin\left(\frac{t}{C^{\frac{2i}{d_{\text{model}}}}}\right), \\ \text{PE}(t, 2i+1) &= \cos\left(\frac{t}{C^{\frac{2i}{d_{\text{model}}}}}\right), \end{aligned} \tag{2}$$

where C is a scaling constant set to 10000.

The positional encoding introduces a unique set of sinusoidal patterns for each position in the sequence, enabling the model to distinguish between different time steps. The choice of $C = 10000$ is based on empirical evidence from the original Transformer model, ensuring that the positional encoding covers a broad spectrum of positional relationships while maintaining numerical stability.

The resulting input $z = x' + \text{PE}$ retains both the feature and temporal information, preparing it for sequential processing by the Transformer encoder.

2) Transformer Encoder: The Transformer encoder processes the input z through multiple layers of multi-head self-attention and feed-forward networks. The self-attention mechanism allows the model to capture both local and global dependencies in the sequence:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{3}$$

where $Q = W_Q z$, $K = W_K z$, $V = W_V z$, and $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}}\times d_k}$ are trainable projection matrices.

The output of the Transformer encoder, $h \in \mathbb{R}^{T\times d_{\text{model}}}$, is pooled to obtain a global representation:

$$h_{\text{pooled}} = \frac{1}{T}\sum_{t=1}^{T} h_t. \tag{4}$$

3) Task-Specific Heads: The global representation h_{pooled} is processed by two task-specific heads:

$$y_{\text{anomaly}} = \sigma(W_{\text{class}} \cdot h_{\text{pooled}} + b_{\text{class}}), \tag{5}$$

$$y_{\text{regression}} = W_{\text{reg}} \cdot h_{\text{pooled}} + b_{\text{reg}}, \tag{6}$$

where σ is the sigmoid activation function for binary classification. $W_{\text{class}}, W_{\text{reg}}, b_{\text{class}}, b_{\text{reg}}$ are trainable parameters for the anomaly detection and capacity estimation tasks, respectively.

C. Split Federated Training

The client-side model processes raw battery data locally, transforming it into intermediate representations (smashed data). To preserve privacy, Gaussian noise is added to the smashed data before transmission, ensuring differential privacy. The noisy smashed data are then sent to the edge server, which completes the forward pass using its layers, computes the multi-task loss, and backpropagates gradients through its own layers and back to the smashed data. These gradients are returned to the EV, which uses them to update its local layers via backpropagation. After completing a fixed number of local epochs, the EV uploads its updated client-side weights, again applying differential privacy by adding Gaussian noise, to the edge server for aggregation. This design reduces the computational load on EVs while safeguarding data privacy and ensuring scalable, distributed training.

Formally, for an EV client k, the local model processes the input data x_k to produce smashed data:

$$h_{\text{client},k} = f_{\text{client}}(x_k; \theta_{\text{client},k}), \tag{7}$$

where f_{client} is the client-side model parameterized by $\theta_{\text{client},k}$. Before transmission, noise is added to ensure (ϵ, δ)-differential privacy:

$$\tilde{h}_{\text{client},k} = h_{\text{client},k} + \mathcal{N}(0, \sigma^2_{\text{smashed}} I). \tag{8}$$

The server completes the forward pass and computes predictions:

$$y_k = f_{\text{server}}(\tilde{h}_{\text{client},k}; \theta_{\text{server}}), \tag{9}$$

where f_{server} is the server-side model parameterized by θ_{server}.

The edge server computes the multi-task loss:

$$\mathcal{L}_k = \alpha \cdot \mathcal{L}_{\text{anomaly},k} + \beta \cdot \mathcal{L}_{\text{regression},k}, \tag{10}$$

where α and β are task-balancing coefficients, and $\mathcal{L}_{\text{anomaly},k}$, $\mathcal{L}_{\text{regression},k}$ are the losses for anomaly detection and capacity estimation, respectively.

During backpropagation, the server computes gradients with respect to the smashed data:

$$\nabla_{\tilde{h}_{\text{client},k}} \mathcal{L}_k = \frac{\partial \mathcal{L}_k}{\partial \tilde{h}_{\text{client},k}}, \tag{11}$$

and returns them to the client. The client then updates its local model:

$$\theta_{\text{client},k} \leftarrow \theta_{\text{client},k} - \eta \nabla_{\theta_{\text{client},k}} \mathcal{L}_k, \tag{12}$$

where η is the learning rate.

After a fixed number of local epochs, the client adds Gaussian noise to its updated weights for additional privacy protection:

$$\tilde{\theta}_{\text{client},k} = \theta_{\text{client},k} + \mathcal{N}(0, \sigma^2_{\text{weights}} I), \tag{13}$$

and sends $\tilde{\theta}_{\text{client},k}$ to the edge server. The edge server aggregates updates from multiple EVs to form a regional model:

$$\theta_{\text{global}} = \frac{1}{K} \sum_{k=1}^{K} \tilde{\theta}_{\text{client},k}. \tag{14}$$

This regional model is transmitted to the cloud server for global aggregation along with updates from other edge servers. After global aggregation, the cloud server redistributes the updated model to all edge servers, which then distribute it to EVs, ensuring model consistency across the network.

To address dataset heterogeneity among EVs and stabilize training, we incorporate a proximal term in the local objective, as in FedProx [28]:

$$\mathcal{L}_{\text{proximal},k} = \mathcal{L}_k + \frac{\mu}{2} \left\| \theta_k - \theta_{\text{global}} \right\|_2^2, \tag{15}$$

where μ controls the contribution of the regularization term and θ_k denotes the local model parameters of client k.

Algorithm 1 offers a comprehensive description of the split federated training process, detailing its implementation at both the EV and ES levels.

Algorithm 1 SMTFL Training

Training phase:

- **Input:** $D_k = \{x_k^{(i)}, y_k^{(i)}\}_{i=1}^{N_k}$ (EV local data)
  T (global rounds), η (learning rate)
  α, β (task weights), μ (regularization coefficient)
  for $t = 1, \ldots, T$ **do**
  - **EV Update:**
    - **Input:** D_k, $\theta_{\text{EV},k}$, θ_{global}
      Compute embeddings:
      $z_k = W_{\text{embed}} \cdot x_k + b_{\text{embed}} + \text{PE}$.
      Extract intermediate features:
      $h_{\text{EV},k} = f_{\text{EV}}(z_k; \theta_{\text{EV},k})$,
      Send $h_{\text{EV},k}$ to ES; Receive $\nabla_{h_{\text{EV},k}} \mathcal{L}_k$;
      Update $\theta_{\text{EV},k}$ with FedProx: $\theta_{\text{EV},k} \leftarrow$
      $\theta_{\text{EV},k} - \eta\big(\nabla_{\theta_{\text{EV},k}} \mathcal{L}_k + \mu(\theta_{\text{EV},k} - \theta_{\text{global}})\big)$
  - **Edge Server (ES) Update:**
    - **Input:** $H = \{h_{\text{EV},k}\}_{k=1}^{K}$
      Forward pass: $y_k = f_{\text{ES}}(h_{\text{EV},k}; \theta_{\text{ES}})$
      Compute loss: $\mathcal{L}_k = \alpha \cdot \mathcal{L}_{\text{class},k} + \beta \cdot \mathcal{L}_{\text{reg},k}$
      Update server parameters:
      $\theta_{\text{ES}} \leftarrow \theta_{\text{ES}} - \eta \nabla_{\theta_{\text{ES}}} \sum_{k=1}^{K} \mathcal{L}_k$

Federated Aggregation phase:

- **Input:** $\{\theta_{\text{EV},k}\}_{k=1}^{K}$
  Output: θ_{global}
  Model aggregation: $\theta_{\text{global}} \leftarrow \frac{1}{K} \sum_{k=1}^{K} \theta_{\text{EV},k}$;
  Broadcast θ_{global} to EVs ;

D. Security and Privacy Analysis

Given the sensitivity of battery operational data, this section establishes our comprehensive security framework.

1) Threat Model: Our framework considers two primary adversary classes:

- **External Network Adversaries**: Third parties capable of intercepting communications between EVs and edge servers, including potential PLC channel eavesdropping despite physical isolation.
- **Honest-but-Curious Edge Servers**: Edge servers that follow the protocol but attempt to infer sensitive information. These adversaries could leverage split learning vulnerabilities, such as analyzing smashed data to reconstruct raw inputs or infer properties of the training data.

2) Security Framework Implementation: Our multi-layered protection addresses these threats:

- **Secure Communication Protocol**: TLS 1.3 with mutual authentication ensures confidentiality, integrity, and authenticity.
- **Differential Privacy for Smashed Data and Gradients**: We apply Gaussian noise to both smashed data and model updates, providing (ϵ, δ)-differential privacy guarantees. This follows DP-SGD [29]:

$$\tilde{g} = \frac{1}{B}\sum_{i=1}^{B} \text{clip}(g_i, C) + \mathcal{N}(0, \sigma^2 C^2 I) \tag{16}$$

  where B is the batch size, g_i the per-sample gradient, C the clipping threshold bounding each gradient's L_2 norm, σ the noise multiplier controlling privacy-utility trade-off, and I the identity matrix. The noise is calibrated to balance privacy and utility.
- **Architectural and Aggregation-Based Privacy**: The split boundary after two Transformer layers ensures smashed data are abstract features rather than raw measurements [27]. Additionally, hierarchical federated aggregation with FedProx provides statistical obfuscation by mixing updates from multiple EVs.

IV. Performance Evaluation

In this section, we evaluate the effectiveness of the proposed framework using the EVBattery dataset [1]. We first introduce the dataset and experimental setup, then present performance under constant and dynamic participation scenarios for both tasks. Subsequently, we provide a detailed complexity analysis of our framework. Finally, we compare and discuss our approach against state-of-the-art (SOTA) methods.

A. EVBattery Dataset

We evaluate our framework using the EVBattery dataset [1], a large-scale real-world benchmark for EV battery diagnostics. The dataset comprises charging snippets collected under diverse conditions—including fast/slow charging and varying temperatures—from three anonymized manufacturers. Each record contains 128-step time-series of eight features (voltage, current, temperature, SOC, etc.) and meta-information (vehicle ID, mileage, snippet index). Binary health labels (0: normal, 1: anomaly) and continuous capacity labels (28.28–46.23 Ah) accompany each sample. A full description is given in Table I; additional dataset specifics are available in the original publication [1].

The dataset was prepared from raw `.pkl` files containing multivariate time-series data and metadata. Sequences were converted into NumPy arrays, and entries with missing or invalid values were discarded. Valid capacity labels (non-zero) were retained as regression targets, while anomaly labels were encoded as binary values (0: normal, 1: anomalies such as lithium plating or thermal runaway). For normalization, all time-series features were standardized per feature (zero mean, unit variance) using a `StandardScaler` fitted exclusively on the training set. Capacity labels were similarly normalized.

TABLE I
EV Battery Dataset Description

Category	Description
Time-Series Data	
Time-steps	128 per charging snippet
Features	V_{avg} (avg. voltage), I_{charge} (current), $V_{max/min}$ (voltage extremes), $T_{max/min}$ (temperature extremes), SOC, timestamp
Meta-Information	
Vehicle ID	Unique identifier
Mileage	At recording time
Snippet index	Position in dataset
Health label	Binary: 1=anomaly, 0=normal
Capacity label	Continuous (28.28–46.23 Ah)

TABLE II
Model Hyperparameters

Parameter	Value
Transformer layers (client/server)	2/4
Attention heads	8
Embedding dimension (d_{model})	64
FFN dimension	128
Dropout rate	0.1
Activation	ReLU
Learning rate	10^{-4}
Batch size	32
Local epochs	20
Optimizer	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Anomaly loss	Binary Cross-Entropy
Capacity loss	Mean Squared Error
Aggregation method	FedProx
Client selection	Poisson process
DP noise scale (σ)	0.3
DP gradient norm (C)	2.0
DP delta (δ)	`1e-5`
Smashed data noise	0.01

TABLE III
Non-IID Sample Distribution Across 20 EVs

EV	Normal	Anomaly	EV	Normal	Anomaly
1	416	192	11	1152	480
2	672	288	12	576	192
3	544	352	13	800	416
4	576	480	14	416	256
5	384	256	15	768	480
6	128	128	16	576	128
7	480	352	17	832	672
8	320	256	18	1888	896
9	608	512	19	1248	1184
10	608	416	20	3552	1856

The cleaned dataset contained no missing values. To preserve real-world conditions, the natural class imbalance was maintained for anomaly detection; in Non-IID splits, each client was guaranteed at least three-batch-size samples per class to prevent extreme skew, without any synthetic resampling. This approach ensures a realistic evaluation of the model under practical data distributions.

B. Implementation & Experimental Results

The architectural details and hyperparameters of the Transformer model are summarized in Table II. The model was

implemented in Python 3.9 using the PyTorch 2.0 deep learning framework. It comprises a stack of six encoder layers, each consisting of an 8-head self-attention mechanism and a feed-forward network with 128-dimensional hidden states. The input data is projected into a 64-dimensional embedding space and enriched with sinusoidal positional encodings to retain temporal information. The loss function combines binary cross-entropy for anomaly detection and mean squared error for capacity estimation to ensure effective training across tasks. For privacy protection, Gaussian differential privacy was applied with noise scale $\sigma = 0.3$, gradient norm bound $C = 2.0$, and $\delta = 10^{-5}$. Additionally, smashed data were perturbed with Gaussian noise of standard deviation 0.01 before transmission. All experiments were conducted on a workstation equipped with an NVIDIA RTX A6000 GPU with 48 GB of memory.

To simulate realistic federated learning scenarios where data distributions vary across clients, we implement a *label-skew* partitioning strategy for Non-IID (Non-Independent and Non-Identically Distributed) experiments. Each EV client is assigned a dominant class (class 0, normal samples) while ensuring a minimum of 3× batch size samples from the minority class (class 1, anomalies). This approach creates heterogeneous local data distributions, reflecting real-world conditions where certain EV clients may observe more frequent anomalies than others. Table III presents the resulting sample distribution across 20 EVs for a representative run, clearly showing the label-skew while maintaining the minimum-sample constraint for both classes.

We validate our evaluation using 5-fold cross-validation. To ensure robust assessment, the folds are constructed with a strict temporal order: within each fold, the training data strictly precedes the test data in time, eliminating temporal data leakage. Additionally, we ensure that no EV client (identified by unique vehicle IDs) is shared between the training and test sets of any fold, preventing cross-client leakage and guaranteeing complete isolation between model development and evaluation data.

Task 1 (Anomaly Detection) is evaluated using accuracy, precision, recall, F1-score, and AUROC (which measures the classifier's ability to distinguish between classes across thresholds):

Task 1 (Anomaly Detection) is evaluated using accuracy, precision, recall, F1-score, and AUROC (which measures the classifier's ability to distinguish between classes across thresholds):

$$\begin{aligned}
\text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN}, \\
\text{Precision} &= \frac{TP}{TP + FP}, \\
\text{Recall} &= \frac{TP}{TP + FN}, \\
\text{F1-Score} &= \frac{2TP}{2TP + FP + FN}.
\end{aligned}$$

Task 2 (Capacity Estimation), formulated as a regression task, is evaluated using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Normalized Root Mean Squared Error (NRMSE):

$$\begin{aligned}
\text{MAE} &= \frac{1}{N}\sum_{i=1}^{N}\left|y_i - \hat{y}_i\right|, \\
\text{MSE} &= \frac{1}{N}\sum_{i=1}^{N}\left(y_i - \hat{y}_i\right)^2, \\
\text{RMSE} &= \sqrt{\text{MSE}}, \\
\text{NRMSE} &= \frac{\text{RMSE}}{y_{\max} - y_{\min}}.
\end{aligned}$$

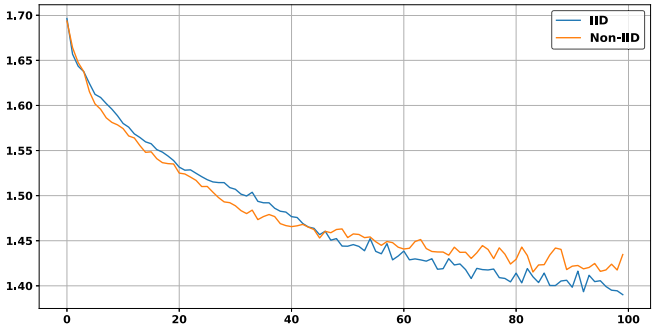


Fig. 2. Global loss over rounds.

We evaluate two experimental configurations: **constant participation**, ensuring a minimum of N EVs per round, and **dynamic participation**, which encompasses two distinct regimes—low participation (simulating off-peak conditions) and high participation (simulating urban peak hours). These scenarios provide a comprehensive assessment of model convergence, scalability, and adaptability under varying participation dynamics.

1) Performance Under Constant EV Participation: Table IV presents the 5-fold cross-validation results. Anomaly detection accuracy remains consistently high (0.988–0.991), while capacity estimation shows stable performance with MAE ranging 0.312–0.338 Ah and NRMSE 0.055–0.059. The low standard deviations (Accuracy: ±0.001, MAE: ±0.013 Ah, NRMSE: ±0.002) confirm that the model does not overfit to specific data subsets and generalizes well.

Figure 2 depicts the progression of prediction model loss across successive training rounds. A steady and continuous decline in loss is evident, culminating in stabilization after approximately 80 rounds. Table V presents a comprehensive performance analysis of the model across different training phases, considering both IID (Independent and Identically Distributed) and Non-IID distributions with a constant participation of 20 EV clients. The results in Table V indicate a consistent improvement in both tasks as training progresses. For battery health anomaly detection, accuracy increases from 0.97 in the early rounds to 0.99 in the final phase, with the F1-score reaching 0.99 (IID) and 0.98 (Non-IID). Convergence is observed around the 80th training round, beyond which performance gains become marginal.

The normalized RMSE (with respect to the total capacity range) decreases from an initial value of 0.07 to 0.05 under

TABLE IV
5-FOLD CROSS-VALIDATION RESULTS (NON-IID)

Fold	Accuracy.	Recall.	Precision	F1-Score	MAE	MSE	RMSE	NRMSE
Fold 1	0.989	0.979	0.991	0.985	0.322	0.415	0.645	0.055
Fold 2	0.991	0.984	0.992	0.988	0.311	0.430	0.656	0.056
Fold 3	0.989	0.978	0.992	0.985	0.329	0.458	0.677	0.058
Fold 4	0.988	0.978	0.990	0.984	0.346	0.525	0.725	0.061
Fold 5	0.990	0.981	0.992	0.986	0.316	0.410	0.640	0.055
Mean ± Std	0.989 ± 0.00	0.980 ± 0.00	0.991 ± 0.00	0.986 ± 0.00	0.325 ± 0.01	0.448 ± 0.05	0.668 ± 0.03	0.057 ± 0.00

TABLE V
MODEL PERFORMANCE ANALYSIS ACROSS TASKS

Tasks	Metric	Early Rounds (1-20)		Mid Rounds (21-80)		Final Rounds (81-100)		Final Results	
		IID	Non-IID	IID	Non-IID	IID	Non-IID	IID	Non-IID
Task 1: Detection	Accuracy	0.97 ± 0.02	0.96 ± 0.03	0.99 ± 0.00	0.98 ± 0.00	0.99 ± 0.00	0.99 ± 0.00	0.99	0.99
	Recall	0.93 ± 0.04	0.91 ± 0.05	0.98 ± 0.00	0.97 ± 0.01	0.98 ± 0.00	0.98 ± 0.00	0.98	0.98
	Precision	0.98 ± 0.03	0.97 ± 0.04	0.99 ± 0.00	0.99 ± 0.00	0.99 ± 0.00	0.99 ± 0.00	0.99	0.99
	F1-Score	0.95 ± 0.03	0.94 ± 0.04	0.98 ± 0.00	0.98 ± 0.00	0.99 ± 0.00	0.98 ± 0.00	0.99	0.98
Task 2: Prediction	MAE	0.43 ± 0.04	0.43 ± 0.04	0.35 ± 0.02	0.35 ± 0.02	0.32 ± 0.00	0.32 ± 0.00	0.31	0.32
	MSE	0.63 ± 0.07	0.63 ± 0.07	0.47 ± 0.04	0.48 ± 0.03	0.41 ± 0.01	0.44 ± 0.01	0.40	0.45
	RMSE	0.79 ± 0.04	0.79 ± 0.04	0.69 ± 0.03	0.69 ± 0.02	0.64 ± 0.01	0.66 ± 0.01	0.63	0.67
	NRMSE	0.07 ± 0.00	0.07 ± 0.00	0.06 ± 0.00	0.06 ± 0.00	0.06 ± 0.00	0.06 ± 0.00	0.05	0.06

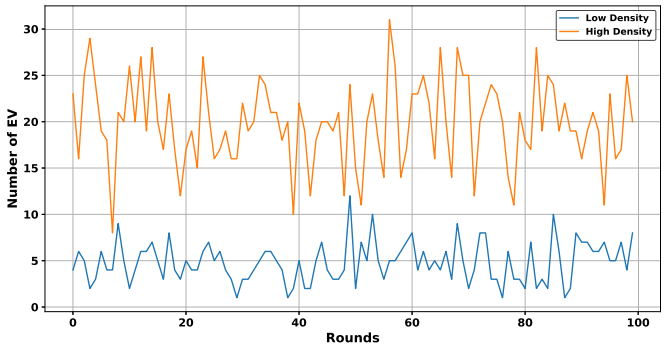


Fig. 3. Number of participating EVs per training round under high-density and low-density scenarios.

IID conditions and 0.06 under Non-IID conditions after convergence (see Table V). The final MAE reaches 0.31 Ah (IID) and 0.32 Ah (Non-IID). Considering the battery capacity range of 28.28 Ah to 46.23 Ah, this represents a relative error of approximately 1.70 % to 1.80 % of the total capacity span. For an average energy consumption of 0.20 Ah/km, this MAE translates to a driving-range uncertainty of only 1.55 km to 1.60 km. These results, stable after approximately 80 rounds and robust to heterogeneous data distributions, confirm the framework's accuracy and reliability for real-time remaining driving range estimation in electric vehicles.

2) Performance Under Dynamic EV Participation: Figure 3 illustrates the number of EVs participating in the training process across different rounds. The high-density scenario maintains a higher and more stable participation (15–30 EVs), while the low-density scenario fluctuates between 3 and 15 EVs, reflecting variations in EV availability.

As expected, a constant and stable participation yields the best performance, as evidenced by the results in Table VI. Although this scenario is rare in practice, it establishes a solid baseline. Under **high-density** conditions, simulating urban environments with heavy traffic, performance remains exceptionally close to the ideal, with an anomaly-detection accuracy of 98.95% (Non-IID) and a capacity-estimation MAE of 0.3394 Ah, confirming that the framework can operate at a near-optimal level in dense EV networks.

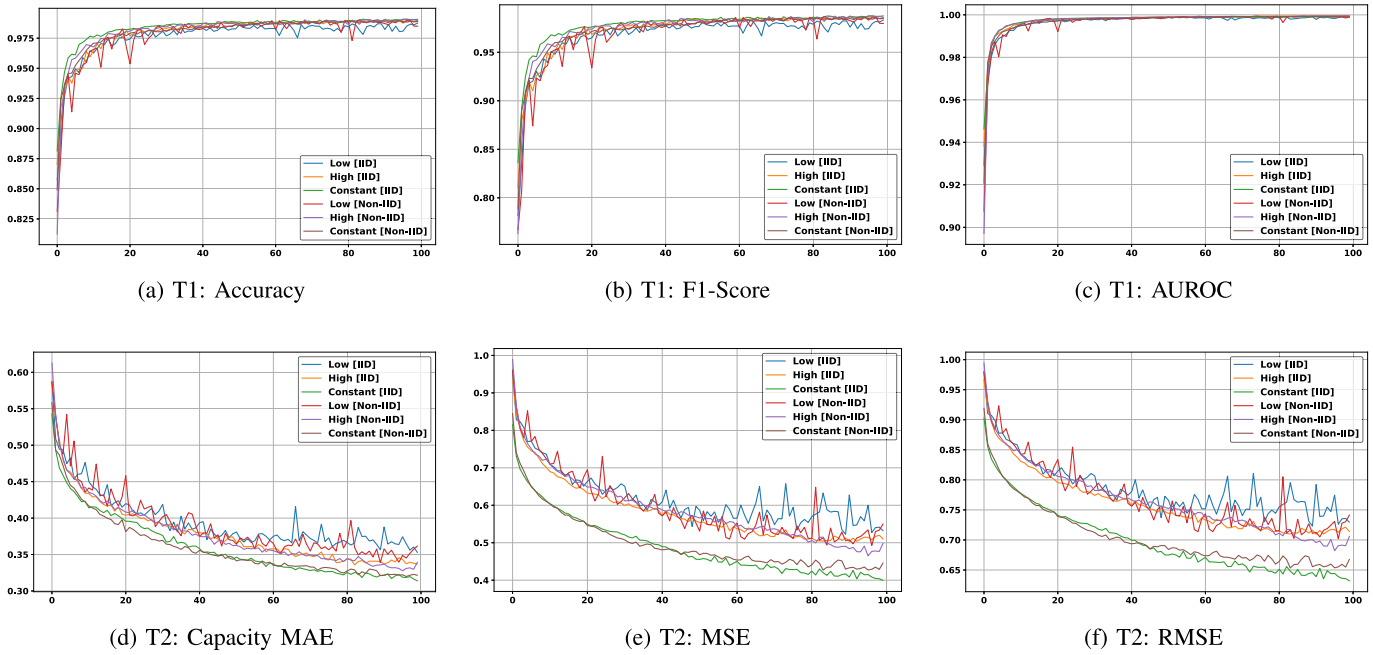
Conversely, the **low-density** scenario, representative of rural areas or off-peak periods, leads to a measurable but limited drop in performance. For anomaly detection, accuracy decreases slightly to 98.50% (F1-score = 97.97%), corresponding to a reduction of 0.58 percentage points. Capacity estimation is more affected, with the MAE rising to 0.3616 Ah (NRMSE = 0.0628), which represents an approximate **15.2%** relative increase compared to the ideal-case MAE (0.3139 Ah). This modest degradation is further illustrated in Figures 4d, 4e and 4f, which plot the evolution of MAE, MSE, and RMSE across the training rounds. The curves confirm that, despite higher round-to-round variability, the error metrics remain bounded and converge to values consistent with those reported in Table VI. Nevertheless, even under this challenging configuration, the results remain excellent and operationally viable: anomaly detection retains an accuracy above 98.5% and capacity estimation exhibits a mean absolute error below 0.37 Ah, which corresponds to an uncertainty of less than 2 km in remaining range (assuming a consumption of 0.2 Ah/km). This demonstrates the robustness of the SMTFL framework in the face of realistic participation fluctuations in electric-vehicle fleets.

C. System Complexity Analysis

We analyze the computational, communication, and memory complexity of the proposed SMTFL framework to assess its

TABLE VI
PERFORMANCE METRICS FOR ANOMALY DETECTION AND CAPACITY ESTIMATION ACROSS DIFFERENT SCENARIOS

		Task 1: Anomaly Detection					Task 2: Capacity Estimation		
EV Participation	**Dist**	**Accuracy**	**Recall**	**Precision**	**F1-score**	**AUROC**	**MAE**	**MSE**	**NRMSE**
High ($\lambda_{EV} = 20$)	IID	0.9888	0.9800	0.9897	0.9848	**0.9994**	0.3369	0.5100	0.0605
	Non-IID	0.9895	0.9816	0.9901	0.9859	**0.9994**	0.3394	0.4983	0.0598
Low ($\lambda_{EV} = 5$)	IID	0.9876	0.9804	0.9860	0.9832	0.9992	0.3530	0.5322	0.0618
	Non-IID	0.9850	0.9747	0.9847	0.9797	0.9989	0.3616	0.5497	0.0628
Constant	IID	**0.9908**	**0.9836**	0.9913	**0.9875**	0.9990	**0.3139**	**0.3994**	**0.0542**
	Non-IID	0.9889	0.9763	**0.9938**	0.9849	**0.9994**	0.3216	0.4458	0.0572



(a) T1: Accuracy (b) T1: F1-Score (c) T1: AUROC

(d) T2: Capacity MAE (e) T2: MSE (f) T2: RMSE

Fig. 4. Performance metrics visualized for different rounds and configurations.

feasibility for deployment in resource-constrained EV environments.

1) Computational Complexity: To evaluate the computational complexity of our SMTFL framework, we calculate the Floating-Point Operations (FLOPs) required for training both the client-side and server-side models.

The client-side model has $P_{\text{client}} = 67{,}520$ parameters (embedding: 576 + 2 transformer layers: $2 \times 33{,}472 = 66{,}944$). The server-side model has $P_{\text{server}} = 134{,}018$ parameters (4 transformer layers: $4 \times 33{,}472 = 133{,}888+$ classification head: 65+ regression head: 65). Each transformer layer consists of multi-head attention (16,640 parameters), feed-forward network (16,576), and two LayerNorm layers (256).

The self-attention mechanism, which dominates computational cost, has quadratic complexity with respect to sequence length L. For each transformer layer, the dominant FLOPs per sample per epoch come from computing the query-key product and the attention-weighted value aggregation, each requiring approximately $2L^2d$ operations per forward pass. Accounting for both forward and backward passes, the total FLOPs per sample per layer is approximately $12L^2d$.

Given $L = 128$, $d = 64$, $N_{\text{samples}} = 768$ (median value across the Non-IID distribution shown in Table III), and $E = 10$ (number of local epochs), the FLOPs per client update are calculated as follows:

- **Client side** (2 transformer layers):

$$\text{FLOPs}_{\text{client}} \approx 2 \times 12L^2d \times N_{\text{samples}} \times E = 1.93 \times 10^{11} \text{ FLOPs.}$$

- **Server side** (4 transformer layers):

$$\text{FLOPs}_{\text{server}} \approx 4 \times 12L^2d \times N_{\text{samples}} \times E = 3.86 \times 10^{11} \text{ FLOPs.}$$

If the entire model were trained locally, the total computational load per client update would be:

$$\text{FLOPs}_{\text{total}} = \text{FLOPs}_{\text{client}} + \text{FLOPs}_{\text{server}} \approx 5.79 \times 10^{11} \text{ FLOPs.}$$

Our split learning design offloads the server-side computation to edge infrastructure, thereby shifting approximately:

$$\frac{\text{FLOPs}_{\text{server}}}{\text{FLOPs}_{\text{total}}} \approx 66.7\%$$

of the computational burden from the EV client to the edge server. This reduction aligns with the proportion of transformer layers executed on the server (4 out of 6 layers). Moreover, without multi-task learning, training two separate models for anomaly detection and capacity estimation would approximately double the computational requirements. Thus, our split multi-task design achieves two key efficiencies: (1) it significantly reduces on-device computation by offloading about two-thirds of the computational load to the edge server, and (2) it eliminates the need for training and maintaining separate task-specific models, further conserving resources. This substantial reduction in on-device computation time and energy consumption makes the framework particularly suitable for resource-constrained EVs during charging sessions.

2) Communication and Energy Overhead: Our framework leverages Power Line Communication (PLC) during charging sessions, utilizing the existing power line infrastructure without requiring additional network resources and eliminating separate wireless transmission energy costs. The communication load per training round consists of uploading intermediate activations (smashed data) and downloading gradients, each approximately 1 MB per batch ($B = 32$, $T = 128$, $d_{\text{model}} = 64$), plus 0.4 MB for client-side model updates, totaling ~ 2.4 MB per round. Over 20 rounds (achieving 96% accuracy and 0.43 MAE), an EV exchanges about 48 MB, orders of magnitude less than transmitting raw battery time-series data ($\sim$ 100 MB). Energy overhead is supplied by the grid; with an embedded GPU (e.g., NVIDIA Jetson AGX Orin, 30 W peak) and $\sim$10 s per round, each round consumes 0.083 Wh. A 20-round session thus uses 1.66 Wh, which is less than 0.02% of a typical fast-charging session (10 kWh). Consequently, the framework introduces negligible communication and energy overhead while preserving the EV's primary function.

D. Comparative Analysis

We evaluate our framework against both centralized and federated baselines to contextualize its performance. Although battery health anomaly detection and capacity estimation have been studied in the literature, to the best of our knowledge, only [1] utilized the EVBattery dataset, enabling a direct comparison. Notably, the authors of [1] employed multiple machine learning methods, providing a valuable opportunity to benchmark our solution against various approaches. However, their work did not incorporate federated learning (FL). Therefore, we implemented FL-based baselines (LSTM, BiLSTM, GRU) to compare our solution against FL methods as well. Figure 5 illustrates the performance comparison of our solution with centralized ML models from [1] for both tasks: AUROC for Task 1 (battery health anomaly detection) and RMSE for Task 2 (battery capacity estimation). These metrics were not selected arbitrarily but were adopted because [1] exclusively relied on them, ensuring a consistent and meaningful comparison.

The AUROC scores demonstrate a clear improvement achieved by the proposed models (MTFL and SMTFL). While traditional methods such as AE, Deep SVDD, and LSTMAD exhibit moderate performance, our models significantly outperform them, reaching an AUROC of 99.9%, marking a substantial advancement in battery health anomaly detection. For the capacity estimation task, evaluated through RMSE, the proposed models also show superior performance. Baseline methods such as Random Forest, XGBoost, and GCNN achieve RMSE values ranging from 1.39 to 1.42, indicating limitations in accurately predicting capacity under complex conditions. In contrast, the proposed models achieve RMSE values of 0.64 and 0.66, respectively, significantly reducing the prediction error. The strong performance of SMTFL can be attributed to its ability to effectively capture multivariate temporal dependencies and feature interactions, leveraging the advanced Transformer-based architecture integrated into the federated learning framework.

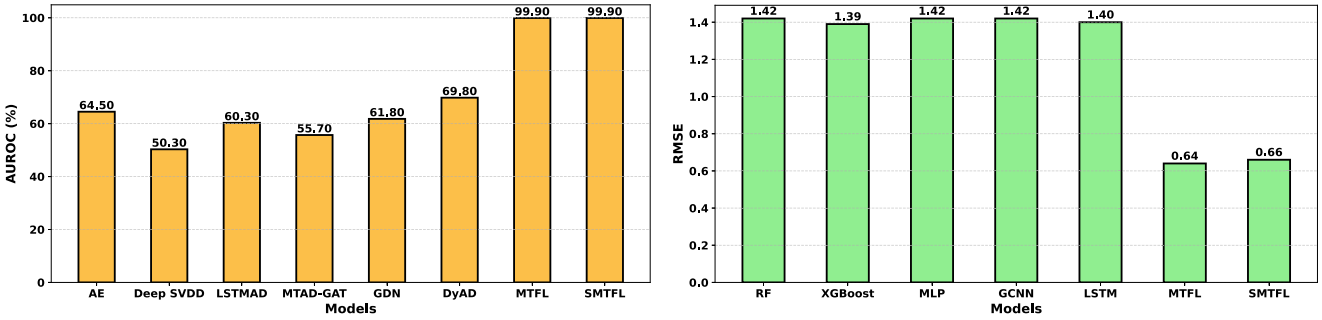
TABLE VII
COMPARISON WITH FL BASELINE MODELS

Model	Accuracy	F1-Score	MAE	RMSE
Baseline models				
FL-LSTM	0.963	0.948	0.478	0.831
FL-Bi-LSTM	0.965	0.951	0.456	0.812
FL-GRU	0.966	0.953	0.463	0.817
Our approach				
MTFL (non-split)	0.988	0.984	0.326	**0.647**
SMTFL (split)	**0.989**	**0.985**	**0.322**	0.668

Table VII summarizes the performance under non-IID data distribution. Baseline models (FL-LSTM, FL-BiLSTM, FL-GRU) yield accuracies around 0.96–0.97 and MAEs around 0.46–0.48. Our FL-Transformer (non-split) notably improves both tasks (accuracy 0.988, MAE 0.326). The proposed SMTFL (split) achieves the best accuracy (0.989), F1-score (0.985), and MAE (0.322), while the non-split variant attains the lowest RMSE (0.647). The slight increase in RMSE for the split model may reflect the added noise from the split process. Our SMTFL framework not only unifies anomaly detection and capacity estimation within a single model while offloading heavy computation to the edge, but also delivers superior performance compared to recurrent baselines, demonstrating the effectiveness of the Transformer architecture with split learning in federated multi-task settings.

V. CONCLUSION AND FUTURE WORK

This paper introduced a Split Multi-Task Federated Learning (SMTFL) framework for joint battery health anomaly detection and capacity estimation in electric vehicles (EVs). The proposed approach combines a Transformer-based encoder with a split federated architecture to achieve state-of-the-art performance while addressing critical challenges in scalability, privacy, and energy efficiency. Experimental results demonstrate that our framework outperforms existing methods, achieving an accuracy of 98.9% for anomaly detection and a MAE of 0.32 for capacity estimation. These results validate the model's capability to capture complex temporal dependencies and feature interactions across both tasks. By enabling EVs to train a single unified model locally and offloading heavy computations to edge servers, the SMTFL framework significantly reduces on-device resource requirements. Moreover, privacy is preserved through differential privacy mechanisms and split



(a) Battery Health Anomaly Detection

(b) Battery Capacity Estimation

Fig. 5. Comparison with centralized ML models.

learning, while training is scheduled exclusively during charging sessions to minimize energy overhead. Collectively, these attributes make the SMTFL framework a practical and efficient solution for large-scale, privacy-aware EV battery diagnostics. Future work will focus on extending the framework to additional battery management system (BMS) tasks, such as predicting battery lifespan and identifying specific component failures. We also plan to explore adaptive split points that dynamically adjust the computation offloading based on device capabilities and network conditions, as well as design more sophisticated multitask models that integrate a broader set of BMS functions without compromising efficiency.

ACKNOWLEDGMENT

The views and opinions expressed are, however, those of the author(s) only and do not necessarily reflect those of European Union or Chips JU. Neither European Union nor the granting authority can be held responsible for them.

REFERENCES

[1] H. He et al., "EVBattery: A large-scale electric vehicle dataset for battery health and capacity estimation," 2022, *arXiv:2201.12358*.

[2] B. Moulik, A. K. Dubey, and A. M. Ali, "A battery modeling technique based on fusion of hybrid and adaptive algorithms for real-time applications in pure EVs," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 3, pp. 2760–2771, Mar. 2023.

[3] I. Saba, M. Ullah, and M. Tariq, "Advancing electric vehicle battery analysis with digital twins in intelligent transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 9, pp. 12141–12150, Sep. 2024.

[4] C. Zhang et al., "An adaptive battery capacity estimation method suitable for random charging voltage range in electric vehicles," *IEEE Trans. Ind. Electron.*, vol. 69, no. 9, pp. 9121–9132, Sep. 2022.

[5] J. Hong, Z. Wang, and Y. Yao, "Fault prognosis of battery system based on accurate voltage abnormity prognosis using long short-term memory neural networks," *Appl. Energy*, vol. 251, Oct. 2019, Art. no. 113381.

[6] D. Li, Z. Zhang, P. Liu, Z. Wang, and L. Zhang, "Battery fault diagnosis for electric vehicles based on voltage abnormality by combining the long short-term memory neural network and the equivalent circuit model," *IEEE Trans. Power Electron.*, vol. 36, no. 2, pp. 1303–1315, Feb. 2021.

[7] P. Li, X. Wu, R. Grosu, J. Hou, M. Ilolov, and S. Xiang, "Applying neural network to health estimation and lifetime prediction of lithium-ion batteries," *IEEE Trans. Transport. Electrific.*, vol. 11, no. 1, pp. 4224–4248, Feb. 2025.

[8] Y. A. Sultan, A. A. Eladl, M. A. Hassan, and S. A. Gamel, "Enhancing electric vehicle battery lifespan: Integrating active balancing and machine learning for precise RUL estimation," *Sci. Rep.*, vol. 15, no. 1, p. 777, Jan. 2025.

[9] K. Das, R. Kumar, and A. Krishna, "Analyzing electric vehicle battery health performance using supervised machine learning," *Renew. Sustain. Energy Rev.*, vol. 189, Jan. 2024, Art. no. 113967.

[10] M. M. Dif, M. A. Bouchiha, A. A. Korba, and Y. Ghamri-Doudane, "Towards trustworthy agentic IoEV: AI agents for explainable cyberthreat mitigation and state analytics," in *Proc. IEEE 50th Conf. Local Comput. Netw. (LCN)*, Oct. 2025, pp. 1–10.

[11] Y. Zheng, Y. Lu, W. Gao, X. Han, X. Feng, and M. Ouyang, "Micro-short-circuit cell fault identification method for lithium-ion battery packs based on mutual information," *IEEE Trans. Ind. Electron.*, vol. 68, no. 5, pp. 4373–4381, May 2021.

[12] G. Ma et al., "Real-time personalized health status prediction of lithium-ion batteries using deep transfer learning," *Energy Environ. Sci.*, vol. 15, no. 10, pp. 4083–4094, 2022.

[13] A. Condon et al., "A dataset of over one thousand computed tomography scans of battery cells," *Data Brief*, vol. 55, Aug. 2024, Art. no. 110614.

[14] J. Zhu et al., "Data-driven capacity estimation of commercial lithium-ion batteries from voltage relaxation," *Nature Commun.*, vol. 13, no. 1, p. 2261, Apr. 2022.

[15] R. Xiong, Y. Zhang, J. Wang, H. He, S. Peng, and M. Pecht, "Lithium-ion battery health prognosis based on a real battery management system used in electric vehicles," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 4110–4121, May 2019.

[16] Z. Deng, L. Xu, H. Liu, X. Hu, B. Wang, and J. Zhou, "Rapid health estimation of in-service battery packs based on limited labels and domain adaptation," *J. Energy Chem.*, vol. 89, pp. 345–354, Feb. 2024.

[17] W. Han, T. Wik, A. Kersten, G. Dong, and C. Zou, "Next-generation battery management systems: Dynamic reconfiguration," *IEEE Ind. Electron. Mag.*, vol. 14, no. 4, pp. 20–31, Dec. 2020.

[18] Y. Liu, J. Ding, Y. Cai, B. Luo, L. Yao, and Z. Wang, "A battery SOH estimation method based on entropy domain features and semi-supervised learning under limited sample conditions," *J. Energy Storage*, vol. 106, Jan. 2025, Art. no. 114822.

[19] P. Kumari and N. Kumar, "Hybrid optimized deep learning approach for prediction of battery state of charge, state of health and state of temperature," *Electr. Eng.*, vol. 106, no. 2, pp. 1283–1290, Apr. 2024.

[20] A. Kant, M. Kumar, and S. Sihag, "Prediction of electric vehicle battery state of health estimation using a hybrid deep learning mechanism," *Int. J. Green Energy*, vol. 22, no. 10, pp. 2065–2078, Jul. 2025.

[21] M. Wu et al., "Federated learning in STAR-RIS aided SAGINs," *IEEE Commun. Standards Mag.*, vol. 9, no. 2, pp. 23–29, Jun. 2025.

[22] P. Abbaraju and S. K. Kundu, "A novel federated & ensembled learning-based battery state-of-health estimation for connected electric vehicles," *IEEE Open J. Intell. Transp. Syst.*, vol. 5, pp. 445–453, 2024.

[23] T. Wang, Z. Y. Dong, and H. Xiong, "Adaptive multipersonalized federated learning for state of health estimation of multiple batteries," *IEEE Internet Things J.*, vol. 11, no. 24, pp. 39994–40008, Dec. 2024.

[24] K. L. Wong, R. Tse, S.-K. Tang, and G. Pau, "Decentralized deep-learning approach for lithium-ion batteries state of health forecasting using federated learning," *IEEE Trans. Transport. Electrific.*, vol. 10, no. 4, pp. 8199–8212, Dec. 2024.

[25] *Road Vehicles—Vehicle to Grid Communication Interface*, Standard ISO 15118, 2022.

[26] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, document RFC 8446, 2018.

[27] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split learning for health: Distributed deep learning without sharing raw patient data," 2018, *arXiv:1812.00564*.

[28] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. 3rd Mach. Learn. Syst. Conf.*, 2020, pp. 429–450.

[29] M. Abadi et al., "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2016, pp. 308–318.